

## Challenge 2

### Switching Game States

#### Exam Objectives Covered

- Determine the process to implement transitions between scenes

### Challenge Overview: Implementing Transitions and State Management

Welcome to the next challenge in your SkiFree-inspired game development journey! While your game's development is progressing, it currently has the feel of a developer prototype. In this session, you will focus on adding transitions for restarting races, switching levels, and quitting the application. This will require some preliminary planning for state management, which will serve as a foundation for future work on the game's UI.

#### Step 1: Create a Race Over Window

Implement a race over window that appears when the player crosses the finish line.

Instructions:

1. Create Race Over UI:
  - Locate the UICanvas prefab in the Prefabs folder.
  - Add and configure UI elements to create a Race Over window that displays when the player finishes the race.
  - Ensure this window can display relevant race information, such as total time and options for the player.

#### Step 2: Implement UI Functions

Add functionality to the Race Over window to handle restarting the race, switching levels, and quitting the application.

Instructions:

1. Reload Current Scene:
  - Implement a function to reload the current scene so the player can attempt the race again.

- Add a button to the Race Over window that triggers this function.
- 2. Navigate to New Level:
  - Design an additional short level as a new scene in Unity.
  - Implement a function to transition from the current scene to this new level.
  - Add a button to the Race Over window that triggers this function.
- 3. Quit Application:
  - Implement a function that initiates a shutdown sequence to quit the application. This should include the capability to call timed events before exiting (e.g., saving data, playing a sound, etc.).
  - Add a button to the Race Over window that triggers this function.

## Step 3: Design Screen Transitions

Create a simple screen transition that can occur between scenes and replays of each level.

### Instructions:

1. Create Transition Animation:
  - Design a simple transition animation (e.g., fade in/out) that will play when switching scenes or reloading the level.
  - Implement this animation using Unity's animation tools or UI system.
2. Integrate Transition Logic:
  - Ensure the transition animation triggers correctly during scene changes and level replays.
  - Adjust the timing and sequence to ensure smooth transitions that enhance the player's experience.

## Step 4: Implement State Management

Plan and implement basic state management to handle these transitions cleanly and efficiently.

### Instructions:

1. Design State Manager:
  - Create a new script, GameStateManager, to handle different states of the game (e.g., active gameplay, race over, transitioning).

## 2. Integrate with Existing Components:

- Ensure that the GameStateManager script interacts with other components (e.g., RaceTimer, UICanvas) to manage transitions and UI updates logically and efficiently.

Once you've completed these steps, your game will have functional transitions and state management, contributing to a polished and streamlined player experience.

Good luck with implementing transitions and state management! This exercise will help you develop skills in UI design, scene management, and state handling in Unity. Happy developing!